Generative AI – 23CSE475

Healthcare Agent for Drug Molecule Generation

Team – 04

Sanjai P G – CB.SC.U4CSE23366

G T Hemanth Kumar – CB.SC.U4CSE23760

Project Overview:

What we built

This project focuses on building a deep learning model that can design new drug-like molecules from scratch. The model uses a character-level LSTM trained on SMILES (Simplified Molecular Input Line Entry System) strings, which are textual representations of chemical structures. By learning patterns and rules from molecular data in the ChEMBL database, the system understands how valid chemical structures are formed. Once trained, it can generate entirely new molecules that resemble real drug compounds, helping researchers explore potential candidates for drug discovery.

Evolution of Workflow:

Phase 1:

- Collected molecular data from the **ChEMBL** database, which contains chemical compounds used in drug discovery.
- Loaded the raw file `chembl_36_chemreps.txt` and extracted only the **canonical SMILES** column using the pandas library.
- Initially limited the dataset to **20,000 entries** to make early experimentation faster and easier.
- Cleaned the data by **removing missing values and duplicate SMILES strings**.
- Applied a **length filter (SMILES < 100 characters)** to remove overly complex molecules that could slow down training.
- Randomly selected **5,000 molecules** with a fixed seed (42) to ensure reproducibility.
- Result: a **clean, balanced dataset of 5,000 SMILES strings** suitable for training the model.

Phase 2: Vocabulary & Tokenization

- Built a character-level vocabulary from the SMILES dataset so the model can understand molecular structures as sequences of characters.
- Added special tokens for sequence control:
  PAD (0) for padding
  SOS (1) for start of sequence
  EOS (2) for end of sequence
- Extracted 41 unique characters from the SMILES dataset to represent atoms, bonds, and structural symbols.
- Total vocabulary size became 44 tokens, which keeps the model efficient while still covering all required chemical symbols.
- During training, SOS was added at the beginning of input sequences, and EOS was added at the end of target sequences, so the model learns when molecules start and end.

Phase 3: Model architecture and design

Component	Setting	Rationale
Embedding dimension	256	Balance between expressiveness and memory
LSTM hidden size	512	Sufficient capacity for SMILES grammar
Number of layers	3	Deep enough for long-range dependencies
Dropout	0.5	Aggressive regularization to prevent memorization
Output projection	Linear(512 → 44)	Maps LSTM output to vocabulary logits
Total parameters	5,813,292	Compact but powerful model footprint

Phase 4: Training Loop Evolution

v1 — Basic Training

- Adam optimizer, fixed learning rate
- No gradient clipping — training unstable on long sequences

- No validation tracking — no early stopping
- Result: oscillating loss, poor generalization

v2 — Stabilized Training (Final)

- AdamW with weight_decay=1e-4 — better generalization via L2 regularization
- Gradient clipping (max_norm=1.0) — prevents exploding gradients on long SMILES
- ReduceLROnPlateau scheduler (factor=0.5, patience=3) — adaptive learning rate
- Best model checkpointing on val_loss — saves optimal weights automatically
- Token-level accuracy tracking — additional signal beyond cross-entropy

Metric	v1 (Initial)	v2 (Improved)
Optimizer	Adam (lr=1e-3)	**AdamW (lr=1e-3, wd=1e-4)**
LR Scheduling	Fixed	**ReduceLROnPlateau**
Gradient Clipping	None	**max_norm=1.0**
Best Model Saving	No	**Yes (val_loss)**
Final Train Loss	0.85 (unstable)	**0.6848**
Final Val Loss	0.90 (unstable)	**0.7014**
Token Accuracy	68%	**76.75%**

Limitations of the Work:

• The current model cannot generate a drug molecule for a specific disease given as input. This is because the training dataset (ChEMBL SMILES data) only contains molecular structures and does not include any information about diseases or drug–disease relationships.

• As a result, the model learns how to generate chemically valid molecules in general, but it does not learn which molecules are useful for treating particular diseases.

• To support disease-based drug generation, the dataset would need to include information linking diseases to drugs or biological targets. Datasets such as DrugBank, BindingDB, or ChEMBL bioactivity data could be used for this purpose. With such data, the model could be modified to take the disease as an input and generate molecules conditioned on that information.

• Another limitation is that if a user provides a completely new or random disease name that was not present in the training dataset, the system will not be able to generate a

meaningful drug molecule. This is because the model can only learn patterns from diseases it has seen during training.

Scope for Future Extensions:

Replacing **SMILES tokenization with SELFIES tokenization**. SELFIES is designed in such a way that every generated sequence automatically represents a valid molecule. Because of this, the model would produce far fewer invalid molecules, and the need for filtering them using RDKit would be greatly reduced.

The current **LSTM-based model** could be replaced with a **Transformer-based architecture**, such as GPT-style models. Transformers are known to perform better on sequence-based tasks because they can capture long-range dependencies more effectively. This is especially useful for representing complex chemical structures in SMILES strings, such as ring closures and branching patterns.

Instead of representing molecules as strings, the system could generate molecules directly as **graphs using Graph Neural Networks (GNNs)**. Since molecules naturally form graph structures (atoms connected by bonds), this approach can represent chemical structures more naturally and avoid some of the limitations of SMILES notation.

Standalone Application:

The current notebook-based implementation can be easily converted into a standalone application so that users can generate drug-like molecules without directly interacting with the training code. This can be done either through a command-line interface or a simple web application.

Command Line Interface (CLI)

A command-line interface can be created to run the molecule generation pipeline directly from the terminal. The CLI would allow the user to specify parameters such as the number of molecules to generate, the sampling temperature, and the top-p value used during generation.

For example, the user could run a command like:

python generate.py --n 100 --temperature 1.0 --top_p 0.9 --start C

This command would generate 100 molecules starting with the character **C**, validate them using RDKit, and then display the valid molecules along with their QED and Lipinski scores.

To build this CLI tool, a few supporting files are required:

- **model.py** – Contains the SmilesLSTM model architecture
- **generate.py** – Handles input arguments, molecule generation, and validation
- **smiles_lstm.pt** – Saved model weights from the training process
- **vocab.json** – Stores the vocabulary mappings used for tokenization

Tech Used and Workflow:

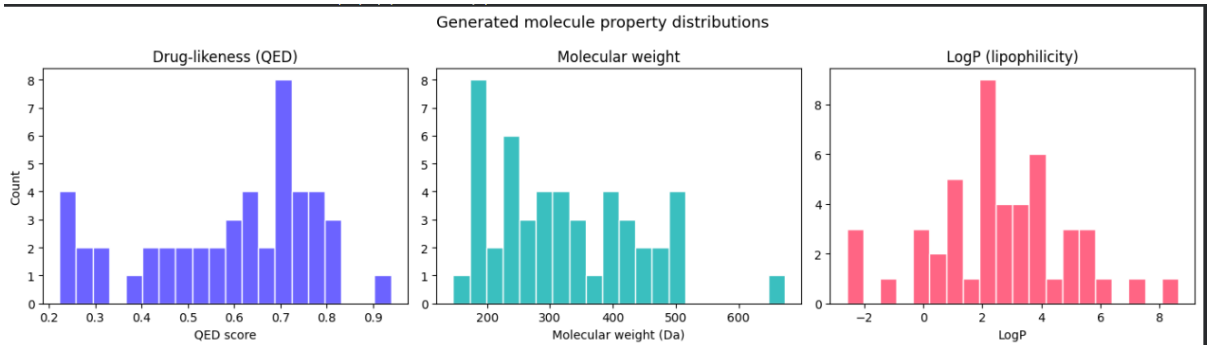
Library / Tool	Role
PyTorch	LSTM model, training loop, tensor operations
pandas	Loading and filtering ChEMBL dataset
RDKit	SMILES validation, Lipinski, QED scoring
matplotlib	Property distribution histograms
Google Colab	GPU-accelerated notebook environment
ChEMBL 36	Source of 2,000 training SMILES strings

Workflow:

- Data ingestion: Read chembl_36_chemreps.txt with pandas, filter length < 100, sample 2,000 molecules
- Tokenisation: Build character vocabulary, assign PAD=0, SOS=1, EOS=2; encode each SMILES as integer indices
- Dataset and loader: SmilesDataset wraps each SMILES as (input= [SOS,...], target= [...,EOS]); DataLoader pads batches dynamically
- Model: SmilesLSTM: Embedding (256) -> LSTM (512 hidden, 3 layers, dropout 0.5) -> Linear(vocab_size)
- Training: AdamW lr=0.001, ReduceLROnPlateau, gradient clipping 1.0, up to 30 epochs, best checkpoint saved
- Generation: Stateful decoding: prime hidden with SOS, feed one character at a time with temperature + top-p sampling

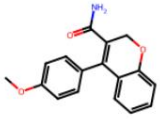
- Validation: RDKit MolFromSmiles gate, canonicalise SMILES, deduplicate, compute QED and Lipinski Rule of Five
- Analysis: Histogram plots of QED, MW, LogP; top 10 molecules ranked by QED; 2D structure grid via RDKit Draw

Output:

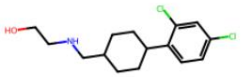



```
Top 10 molecules by QED score:
#    QED     MW       LogP    Ro5    SMILES
-----------------------------------------------------------------
1    0.939   281.31   2.37    ✓      COc1ccc(C2=C(C(N)=O)COc3ccccc32)cc1
2    0.81    302.25   3.85    ✓      OCCNCC1CCC(c2ccc(Cl)cc2Cl)CC1
3    0.805   284.16   5.31    ✗      CC(Nc1cc(Cl)cc(Cl)c1)c1ccc(F)cc1
4    0.8     247.29   2.09    ✓      O=C(CCCc1ccccc1)ON1CCCC1=O
5    0.783   338.45   3.71    ✓      NC(=O)c1ccc(-c2ccc(OCCCN3CCCCC3)cc2)cc1
6    0.783   331.8    3.51    ✓      CC(C)Nc1ccc(Cl)cc1C(=O)Nc1ccc(C(N)=O)cc1
7    0.778   265.35   1.95    ✓      Cc1cc(C(CCO)C(N)=O)c(O)c(C(C)(C)C)c1
8    0.767   249.31   3.93    ✓      CC1OC(c2ccccc2)Cc2ccc(C#N)cc21
9    0.755   315.33   0.11    ✓      Cc1cc([C@@H]2C(=O)C(=O)N=C2NC(=O)[C@@H]2CCN2C)ccc1O
10   0.738   234.23   1.67    ✓      Nc1nc(CO)c(O)cc1-c1ccc(F)cc1
```

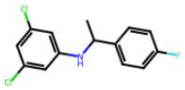
```
── Metrics (valid + unique molecules)
  QED     mean=0.586   max=0.939
  MW      mean=320.8    range=[146, 673]
  LogP    mean=2.60
  Ro5 pass rate: 37/48 = 77%
```



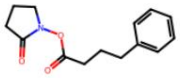
QED=0.939



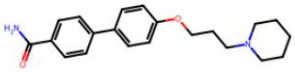
QED=0.81



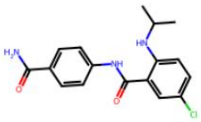
QED=0.805



QED=0.8



QED=0.783



QED=0.783